Capstone Project: OpenMarket Corp. Software Architecture

Multi-platform E-Commerce System Design

Team presentation for Software Architecture 1 course

Made with GAMMA

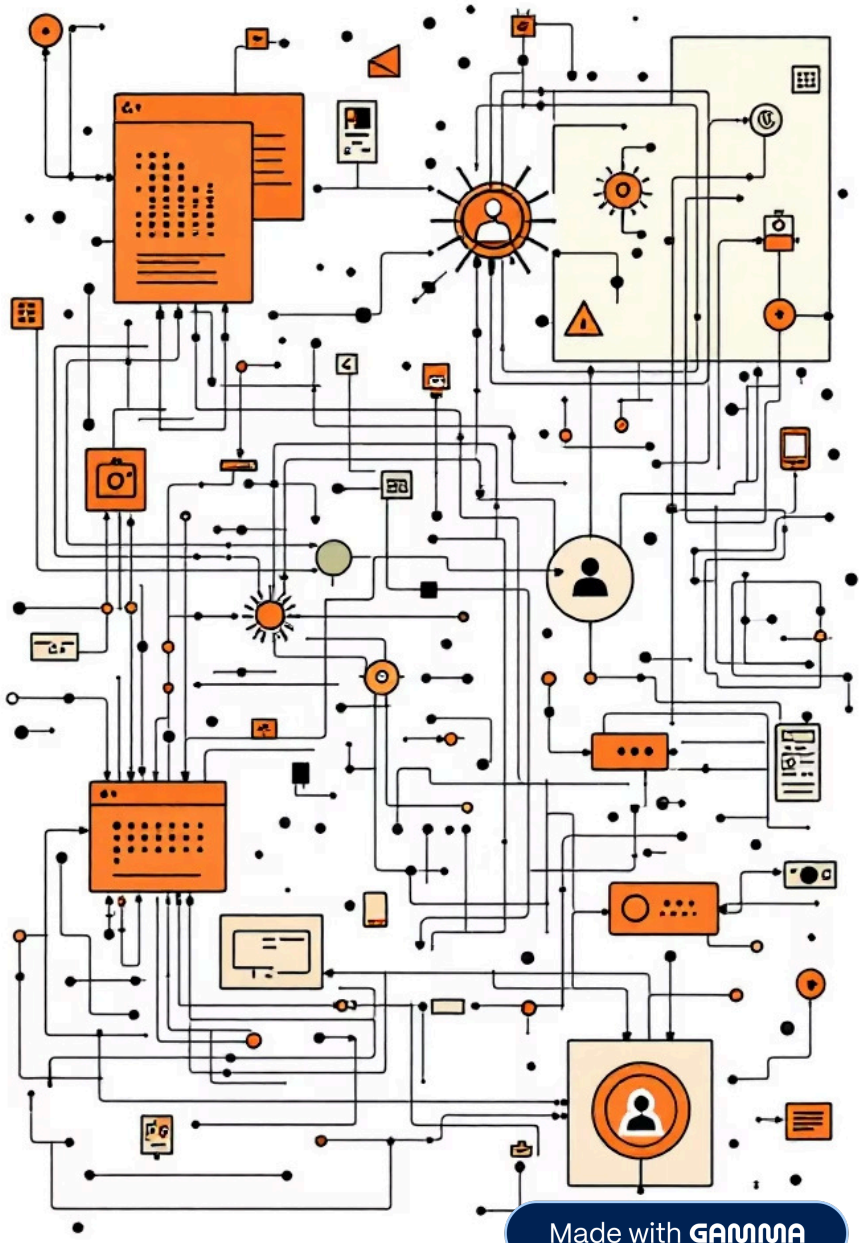
Project Overview & Objectives

Project Mission

Design a robust, scalable architecture for a high-traffic e-market platform that handles complex transactions while maintaining security and reliability.

Core Deliverables

- `ARQUITECT.md` - Central documentation hub
- ADR Repository - Architecture Decision Log
- Empty class structure for metric validation
- Integration framework for third-party services



Made with GAMMA

Architectural Drivers: The "Why"

Elasticity

System must handle traffic spikes during peak seasons and promotional events without performance degradation.

Availability

24/7 uptime guarantee, especially critical during high-revenue periods like holiday shopping seasons.

Security

PCI-DSS compliance for financial data protection and secure transaction processing.

Additional Considerations: Maintainability for long-term evolution, Testability for quality assurance, and Deployability through CI/CD pipelines.

Made with GAMMA

Architectural Style & Strategy

Chosen Architecture

Microservices

Primary Justification

This architectural style directly addresses our elasticity and extensibility drivers by enabling independent scaling of components and clear separation of concerns.

Trade-offs & Mitigations

Complexity: Increased system complexity managed through comprehensive documentation and ADR tracking.

Latency: Potential inter-service communication delays mitigated by asynchronous messaging patterns and caching strategies.

Data Consistency: Eventual consistency patterns ensure reliability while maintaining performance.

Made with GAMMA

Architecture Styles Evaluation

Style	Advantages	Disadvantages	Alignment
Monolithic Architecture	Simple deployment and development	Limited scalability and single point of failure	Low
Microservices Architecture	Independent scaling, fault isolation, service autonomy	Higher operational complexity	High
Event-Driven Architecture	Loose coupling, asynchronous processing	Data consistency challenges	Medium

Decision Outcome

Selected Architecture: Microservices

Complementary Pattern: Event-Driven Communication

Made with GAMMA

Why Microservices + Event-Driven?

Microservices

+

Event-Driven

Elasticity

Independent horizontal scaling of services

Availability

Failure isolation to maintain system operation

Security

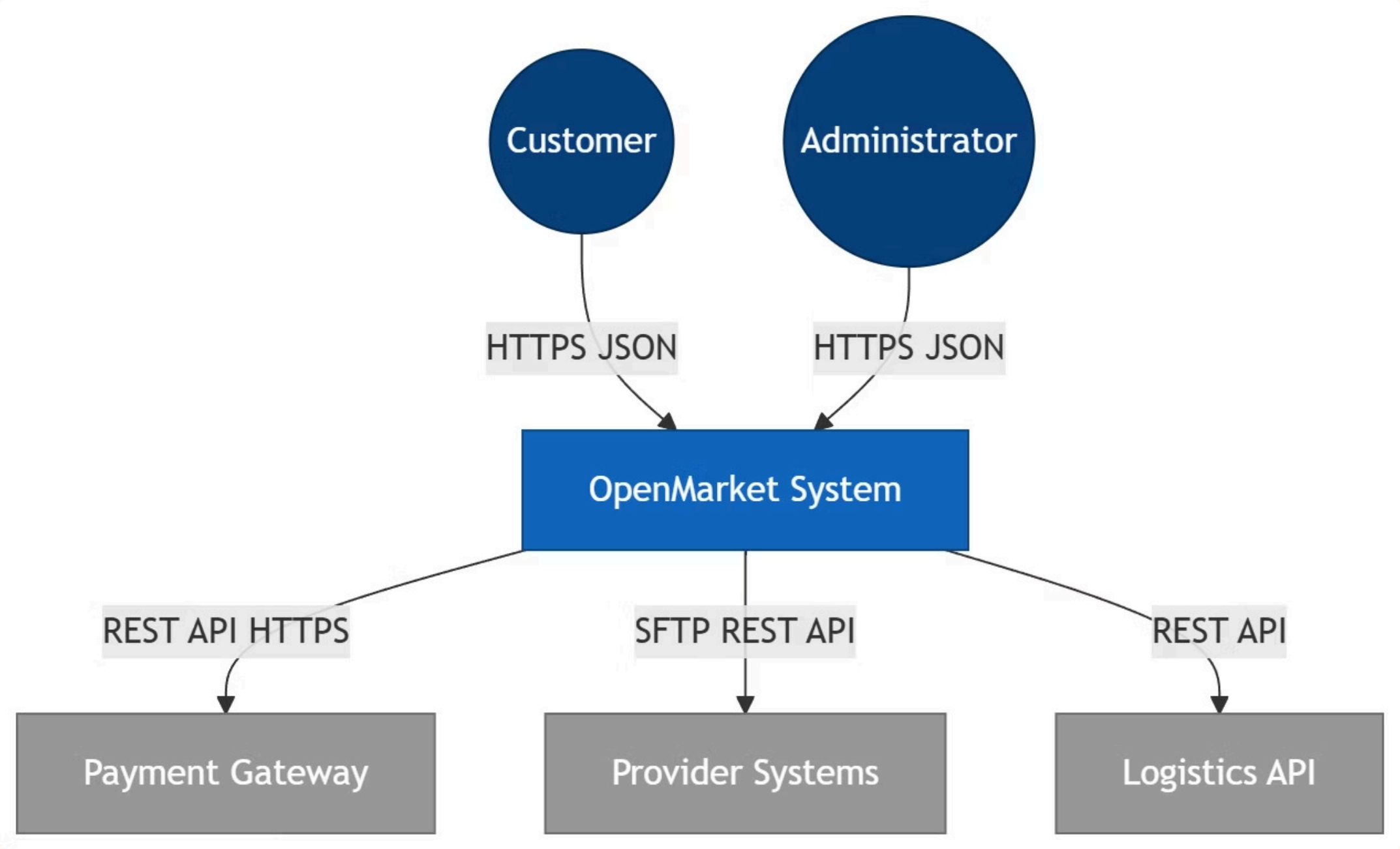
Separation of sensitive domains (auth & payments)

Deployability

Independent deployment cycles supporting CI/CD

Made with GAMMA

C4 Model - Level 1: System Context



01

Customer/Admin to System

Communication via **HTTPS** ensuring secure data transfer. Data format is **JSON** for API consumption or HTML for web rendering.

02

System to Payment Gateway

Synchronous integration via **REST API** (e.g., Stripe/PayPal). Critical for **Security (PCI-DSS)** compliance as per Román's Req 3.4.

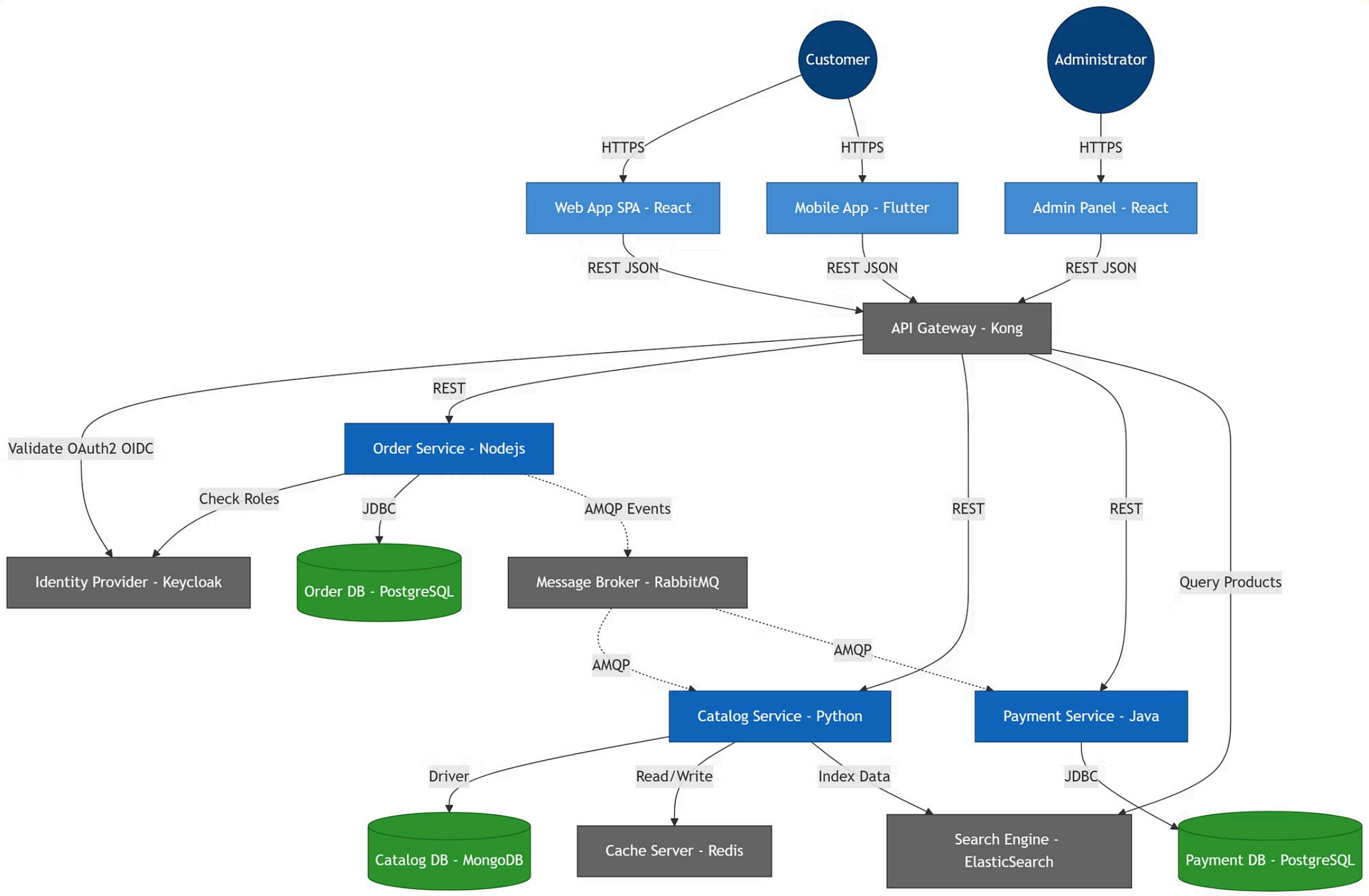
03

System to Provider Systems

Hybrid integration. **SFTP** for batch CSV file processing (Req 1.1) and **REST** for real-time catalog updates, ensuring **Interoperability**.

Made with GAMMA

C4 Model - Level 2: Containers



Identity Provider (Keycloak):

Centralizes Security (Req 3.4). Handles user authentication and issues JWT tokens. Uses OIDC (OpenID Connect) protocol.

Cache Server (Redis)

In-memory data store. Drastically improves Responsiveness (Req 2.1) by caching product listings and prices. Uses TCP protocol.

Search Engine (ElasticSearch)

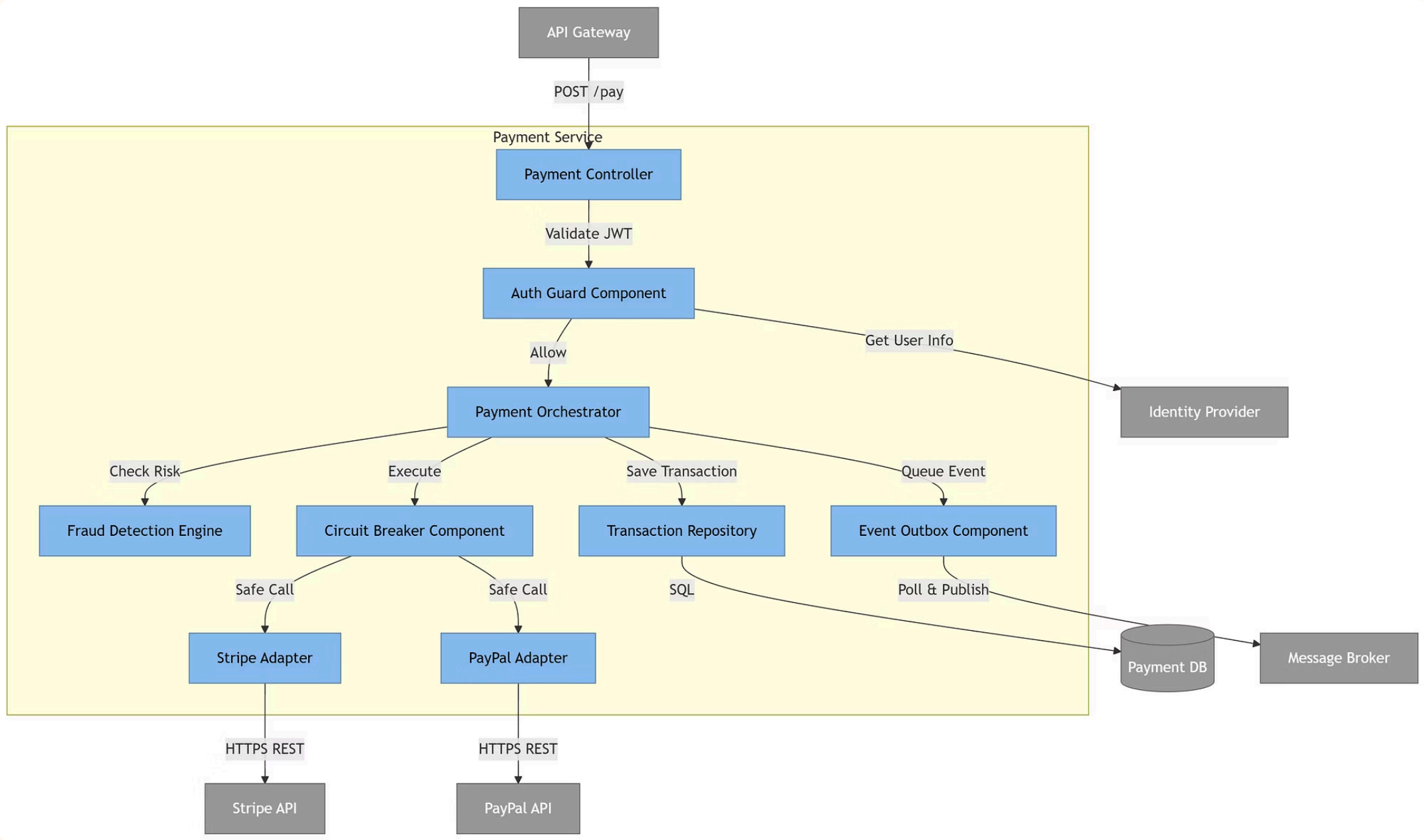
Dedicated indexer. Allows complex filtering and text search without overloading the main database.

Database per Service

Polyglot persistence. **PostgreSQL** for transactional integrity (Orders/Payments) and **MongoDB** for flexible catalog data. Ensures high **Data Consistency** (Req 2.5) for financial transactions.

Made with GAMMA

C4 Model - Level 3: Components (Payment Service)



- **Circuit Breaker Component:** Implements the Circuit Breaker Pattern (e.g., using Resilience4j). If external payment APIs are down, it fails fast, protecting the system from thread blocking, ensuring high Availability (Req 3.3).
- **Fraud Detection Engine:** Analyzes transaction metadata (IP location, amount) before processing. Increases Security beyond just authentication.
- **Event Outbox Component:** Ensures that a database transaction and a message broker publication happen atomically. Guarantees Data Consistency (Req 2.5) — "The payment was saved AND the notification was sent."
- **Auth Guard:** Communicates with the Identity Provider via REST to validate tokens on every critical request.

Made with GAMMA

Architectural Metrics & Class Analysis

I

Instability

Measure of how dependent a component is on other components (0-1 scale)

A

Abstraction

Measure of abstract vs concrete elements in the design (0-1 scale)

D

Main Sequence Distance

Calculated as `D = |A + I - 1|`

Component Evaluation Results

[Insert Metrics Data] - Component measurement results from class diagram analysis

Balanced Component

If `D ≈ 0`: Design is well-balanced with appropriate levels of abstraction and stability.

Requires Refactoring

If `D` is high: Component needs restructuring to improve maintainability and reduce coupling.

Made with GAMMA

Architectural Metrics Evaluation: Payment Orchestrator (Level 3)

Metric	Value	Context
Abstraction (A)	0.8	Based on 4 key interfaces: Fraud, Circuit Breaker, Repository, Outbox.
Instability (I)	0.8	Identified 4 efferent (outgoing) and 1 afferent (incoming) dependencies.
Main Sequence Distance (D)	0.6	Calculated as $\lvert A + I - 1 \rvert$.

Technical Analysis

The Payment Orchestrator component exhibits a Main Sequence Distance (D) of 0.6, positioning it in the **'Zone of Uselessness'** on the A vs I graph. This indicates a significant imbalance: the component is **over-abstracted** for its current level of instability.

This suggests a risk of **over-engineering**, where too many interfaces have been created for a flow that primarily depends on external services, which themselves are prone to frequent changes. The point $(I : 0.8, A : 0.8)$ lies considerably far from the ideal 'Main Sequence' diagonal.

Conclusion & Recommendation

> The design, while highly flexible, introduces unnecessary maintenance complexity. We recommend evaluating the consolidation of interfaces to bring the component closer to the 'Main Sequence' (D near 0) for optimal balance between abstraction and instability.

Made with GAMMA

Repository Structure & ADRs

ADR Repository

Documented architecture decisions with rationale, context, and consequences tracked in `/adrs` folder.

Source Structure

Organized `/src` directory following chosen architectural pattern with clear separation of concerns.

Central Documentation

`ARQUITECT.md` synthesizes all analysis, diagrams, and decisions into unified reference.

DevOps & Integration Workflow

CI/CD pipeline automates testing, builds, and deployment while ADRs provide historical context for future maintainers.

Made with GAMMA

Conclusion & Team Credits

Project Summary

Our architecture successfully balances immediate performance requirements with long-term maintainability through strategic pattern selection and comprehensive documentation practices.

Team Members

- **Román De Giacomo De Luca** - Requirements & Drivers Analyst
- **Alejandro Gonzales Jesus** - Architectural Style Designer
- **Cristian Galeano** - Visual Architect - C4 Model
- **Said Acosta Cepeda** - Component & Metrics Analyst
- **Sebastián Pereyra Beltran** - DevOps & Integration Lead

Final Deliverables

- Complete C4 model documentation
- Architecture Decision Records
- Component metrics analysis
- Deployable code structure

Thank you for your attention

We are ready to answer your questions about our architectural decisions and design trade-offs.

Made with GAMMA